פרויקט מסכם בקורס נושאים מתקדמים בשפת C

המחלקות להנדסת חשמל והנדסת מחשבים

סמסטר אביב, תשפ"ה

מבוא

עליכם לפתח מערכת לניהול עמדות טעינה של חברת EasyCharge עם פריסה ארצית בכל הארץ.

המערכת תפותח בשפת C ותשתמש במבני נתונים אבסטרקטיים כגון רשימות מקושרות, עצים, ותור. המערכת תתבסס על Data base של תחנות טעינה ורכבים הרשומים לשירות.

תיאור כללי של המערכת: המערכת תבצע טעינה של נתונים מקבצים בתחילת התוכנית, תאפשר הוספת/מחיקת עמדות טעינה, רכבים, הפקת דוחות, ותבצע שמירה של הנתונים לאחר עדכונים וביציאה מהמערכת.

הנחיות כלליות

יש להקפיד על שימוש בשמות משתנים ושמות פונקציות כפי שהוגדרו בעבודה. במידה וישנם משתנים או פונקציות ששמם לא הוגדר, ניתן להגדירם כרצונכם. **אין להשתמש במשתנים גלובליים.**

o **שאלות לצוות הקורס** יש להפנות באמצעות קובץ השאלות **בלבד** ולא באופן פרטי – השאלות תתרכזנה בהבנת המשימה. שאלות הקשורות לאופן המימוש או יישום הסטודנט לא יקבלו מענה. יש לוודא כי לא קיימת שאלה דומה בקובץ לפני כתיבת השאלה חדשה.

https://docs.google.com/spreadsheets/d/1boRexFQxxrEURxmk0WZtsil4k7yVLtZDVmt9WFyyQ8U/edit?usp=sharing

תיאור המערכת

פעולות טעינה ושמירה בעת פתיחת המערכת וסגירתה

בעת פתיחת המערכת יטענו נתונים מתוך הקבצים לתוך מבני נתונים מתאימים (ראו קבצים מצורפים ומבני נתונים שמוגדרים בהמשך):

1. מאגר **תחנות** טעינה **Stations.txt** – **לעץ חיפוש בינארי** לפי id. מספר ה-id הינו חד-חד ערכי.
2. מאגר **עמדות** טעינה **Ports.txt** – לכל תחנת טעינה יש מספר של עמדות טעינה. הנתונים של העמדות יטענו **לרשימה מקושרת חד כיוונית**. רכבים הנמצאים בטעינה יצויינו באמצעות מספר הרישוי שלהם.
3. מאגר של **לקוחות** של החברה **Cars.txt** – תיטען **לעץ חיפוש בינארי** לפי מספרי רישוי של רכבים.
4. רשימת **הרכבים שנמצאים כעת בתור** לטעינה בתחנות **LineOfCars.txt** – **לתור queue בהתאם לתחנה בה כל רכב נמצא** (המידע השמור בקובץ התור הינו מספרי הרכבים ומספרי תחנות הטעינה בה הם נמצאים בתור לפי סדר ההגעה). כלומר, יש לדאוג לטעון את המידע לתור שמשויך לתחנות המתאימות.

בעת סגירת המערכת וכן לאחר כל שינוי בנתונים יעודכנו הקבצים הרלוונטיים:

1. מאגר התחנות **Stations.txt** – תחנות עדכניות במערכת
2. מאגר של עמדות **Ports.txt** – נתונים עדכניים של עמדות הטעינה המשויכות לכל תחנה
3. מאגר לקוחות הרכבים **Cars.txt** העדכני
4. הרשימה העדכנית של הרכבים שנמצאים בתורים לעמדות **LineOfCars.txt** לפי סדר התחנות

תיאור כללי של פעולת המערכת

1. המערכת תתבסס על תפריט ממנו תיבחר הפעולה המתבקשת ע"י המשתמש.
2. במהלך האינטראקציה עם המשתמש, כל הקלטים יבדקו, ואם הקלט אינו תקין, תוצג הודעה מתאימה למשתמש שתאפשר לו להכניס את הקלט מחדש או תאפשר לחזור לתפריט הראשי לבחירת פעולה חדשה.
3. יש לדאוג לאתחל משתנים.
4. יש לדאוג לכך שבכל פעולה יתבצע עדכון של הנתונים בכל המבנים הרלוונטיים וכך שתתקיים נורמליזציה של Data.

דרישות מימוש המערכת

הגדרת מבנים

מבנה של תאריך – Date יכלול את השדות הבאים:

- **Year:** שנה - מספר שלם חיובי בן 4 ספרות - שנה תקינה = החל משנת 1900 ועד השנה הנוכחית
- **Month**: חודש - מספר שלם חיובי, בין 1-12
- **Day**: יום - (מספר שלם חיובי)
- **Hour**: שעה – בפורמט של 24 שעות, מספר שלם בין 0 ל-23
- **Min**: דקות – מספר שלם בין 0-59

מבנה של רכב – Car יכלול את השדות הבאים:

- **nLicense:** מספר רישוי רכב (מחרוזת בת 9 ספרות הכוללת '\0') – תכיל ספרות בלבד והמספר יהיה חד- חד ערכי
- **portType:** קצב טעינה מתאים { FAST, MID, SLOW } enum
- **totalPayed:** התשלום הכולל עד היום – double
- **pPort**: מצביע לעמדה (**Port**) בזמן הטעינה. אם לא בטעינה אז NULL
- **inqueue**: 0 – לא בתור/בטעינה, 1 – בתור

מבנה עלה בעץ חיפוש בינארי של מאגר מכוניות – tCar יכלול את השדות הבאים:

- מצביע למכונית מטיפוס **Car**
- מצביע למכונית **right**
- מצביע למכונית **left**

מבנה צומת בתור של מכוניות בתחנה – carNode יכלול את השדות הבאים:

- מצביע למכונית מטיפוס **Car**
- מצביע למכונית הבאה בתור **next**

מבנה מנהל של תור מכוניות – qCar יכלול את השדות הבאים:

- מצביע לתחילת התור **front** מטיפוס **carNode**
- מצביע לסוף התור **rear** מטיפוס **carNode**

מבנה של עמדה – Port יכלול את השדות הבאים:

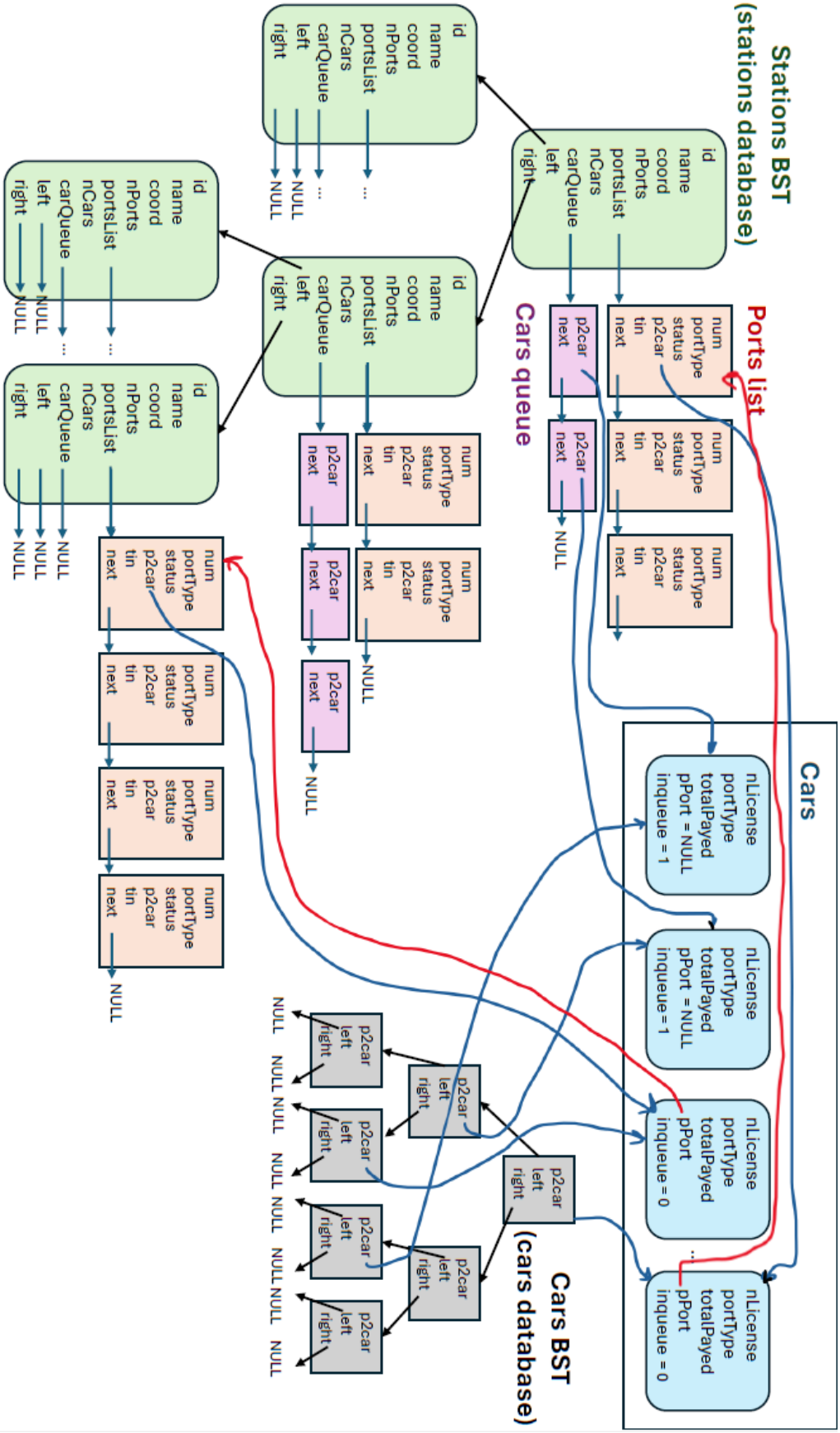
- **num:** מספר עמדה - מספר שלם
- **portType:** קצב טעינה של העמדה enum {FAST, MID, SLOW}
- **status:** **1** – occupied, **2** – free, **3** – out of order
- **p2car:** מצביע לרכב מטיפוס **Car** הנטען בעמדה. אם לא מחובר רכב יאותחל ל- NULL
- **tin:** זמן תחילת טעינה מטיפוס **Date**
- **next:** מצביע לעמדת הטעינה הבאה ברשימה

מבנה תחנה – Station יכלול את השדות הבאים:

- **id:** מספר שלם חד-חד ערכי
- **name:** שם של התחנה - מצביע למחרוזת דינאמית
- **nPorts:** מספר העמדות בתחנה – מספר שלם
- **coord:** קואורדינטות מיקום התחנה – מבנה הכולל שדות x,y שכל אחד מהם double
- **portsList:** ראש רשימה מקושרת של כל העמדות מטיפוס **Port** בתחנה
- **nCars:** מספר הרכבים בתור
- **carQueue:** מבנה מנהל של תור מטיפוס **qCar**
- מצביע לתחנה **right**
- מצביע לתחנה **left**

יש להגדיר מבני מנהלים מתאימים ולהתייחס אליהם כאל ADTs (תור, עץ, רשימה מקושרת), לבנות קבצי קוד ו-headers מתאימים לטיפול במבני הנתונים. שימו לב, מבנה מנהל לתור הרכבים כבר הוגדר עבורכם.

תרשים מבני הנתונים של המערכת



פונקציות למימוש

יש לבנות תוכנית מודולרית וקריאה, לכן יש לפרק את הפעולות השונות לתתי פונקציות במידה וישנו צורך.

יש להשתמש בסביבה מרובת קבצים למימוש מבני הנתונים השונים.

פונקציות עזר שהן חובה

loadFiles – טעינת כל נתוני הקבצים למערכת בהתאם למבני הנתונים שהוגדרו. שדות שלא ניתן להם ערך בקובץ יאותחלו ל-NULL/0 בהתאם לטיפוס. **פעולת הטעינה תתרחש בעת פתיחת המערכת - לפני הצגת התפריט.**

updateFiles – שמירת הנתונים העדכניים לקבצים לאחר כל פעולת עדכון של נתונים או בזמן יציאה מהתוכנית.

searchStation – הפונקציה תאפשר חיפוש תחנה לפי שם התחנה, id או מרחק ממיקום התחנה למיקום המשתמש. הטיפול בסוגים השונים של הקלט יתבצע באמצעות מערך של מצביעים לפונקציות (טיפול ב- id או מרחק או שם התחנה). הפונקציה **searchStation** תחזיר את המצביע לתחנה.

searchCar –פונקציה שמקבלת מספר רישוי ומחזירה מצביע לרכב מטיפוס **Car** בעל מספר רישוי מבוקש

displayError – פונקציה להדפסת הודעות שגיאה – מקבלת קוד שגיאה ומדפיסה הודעה מתאימה

assignCar2port – פונקציה לקישור רכב לעמדת טעינה ←, וקישור עמדת טעינה לרכב ←. הפונקציה מקבלת מצביע למכונית ומצביע לעמדת טעינה ומבצעת את הקישור ביניהם. הפונקציה מעדכנת את כל השדות הרלוונטיים להשלמת פעולת תחילת הטעינה – למשל עדכון status של עמדת הטעינה, עדכון זמן תחילת הטעינה tin וכו'.

פעולות תפריט

יש לבנות פונקציית **Menu** המדפיסה תפריט למשתמש בהתאם לרשימת האפשרויות הבאה (פונקציית התפריט מדפיסה בלבד – לא מקבלת ולא מחזירה ערכים):

פעולה 1 בתפריט: **Locate nearest station** – מציאת תחנה לטעינה **לפי מרחק.**

יש לכתוב פונקציה בשם **locNearSt**. הפונקציה מקבלת את המבנה המנהל של BST של תחנות הטעינה. הפונקציה מבקשת מהמשתמש קואורדינטות מיקום שבהן הוא נמצא – x ו- y. הפונקציה מוצאת את התחנה הקרובה ביותר בקו אווירי למיקום המשתמש ומדפיסה את הנתונים הבאים של התחנה שמצא: שם ו-id של התחנה, מרחק הרכב מהתחנה, מספר עמדות פעילות בתחנה (לא תקולות) ופירוט סוגי עמדות הטעינה (SLOW, MID, FAST), מספר עמדות תפוסות ברגע זה, ומספר מכוניות הממתינות בתור לטעינה.

פעולה 2 בתפריט: **Charge car** – הכנסת מכונית לעמדת טעינה, ואם אין עמדות פנויות אז לתור.

יש לכתוב פונקציה בשם **chargeCar**. הפונקציה מקבלת את המבנה המנהל של BST של תחנות הטעינה ואת המבנה המנהל של BST הרכבים. הפונקציה תקלוט מהמשתמש מספר רכב ומידע על התחנה (id או שם). במידה והרכב לא נמצא בעץ המחזיק את מאגר הרכבים, עליכם לקלוט מהמשתמש את כל הפרטים הרלוונטיים של הרכב ולהכניס את הרכב ל-BST לפי מספר הרישוי. יש לבדוק את הקלט באופן יעיל, כך שהמשתמש לא יצטרך להקליד את הכל מחדש אם טעה בהקלדת אחד הנתונים.

יש לוודא שיש עמדת טעינה תקינה שיכולה להתאים לרכב (SLOW, MID, FAST). יש לחבר לעמדת טעינה המתאימה לסוג הטעינה של הרכב בתחנה המבוקשת. במידה ואין עמדת טעינה מתאימה פנויה, עליכם להכניסו לסוף התור. יש לעדכן את כל השדות הרלוונטיים הנדרשים לניהול תקין של מערכת המידע. יש להשתמש ב- time.h לקליטת הזמן הנוכחי בזמן תחילת הטעינה. ניתן להיעזר בצ'אט כדי ללמוד כיצד להשתמש בפונקציות של הספרייה. יש להדפיס הודעות מתאימות למשתמש.

פעולה 3 בתפריט: **Check car status** – בדיקת זמן טעינה של רכב לפי מספר רכב.

יש לכתוב פונקציה בשם **checkCarStatus**. הפונקציה מקבלת מבנה מנהל של עץ הרכבים ומבנה מתנהל של התחנות. הפונקציה קולטת מהמשתמש מספר רכב ובודקת האם הרכב נמצא בתור, בטעינה או כלל אינו נמצא בתחנת טעינה.

במידה והרכב בטעינה, הפונקציה תדפיס את שם התחנה בה הוא נמצא, את מספר העמדה ומספר דקות הטעינה של הרכב.

במידה והרכב בתור, הפונקציה תדפיס את שם התחנה בה הוא נמצא, ואת מיקומו בתור בהתחשב בסוג העמדה המתאים לרכב זה – טעינה מהירה/בינונית/איטית.

בשאר המקרים, התוכנית תדפיס הודעה מתאימה.

פעולה 4 בתפריט: **Stop charge** – תפסיק טעינה של רכב.

יש לכתוב פונקציה בשם **stopCharge**. הפונקציה מקבלת מבנה מנהל של עץ הרכבים ומבנה מתנהל של התחנות.

הפונקציה תקלוט מהמשתמש את מספר הרכב ותנתק אות מהטעינה במידה והוא מחובר.

לאחר מכן, הפונקציה תקדם את הרכב הבא בתור בתחנה במידה וישנו רכב עם קצב טעינה מתאים.
הפונקציה תעדכן את הסכום ששולם לפי 1.2 שקלים לדקת טעינה (יש להשתמש ב- time.h לקליטת הזמן הנוכחי שיהווה זמן סיום הטעינה), ותעדכן את כל השדות הרלוונטיים הנדרשים לניהול תקין של מערכת המידע. יש להדפיס הודעות מתאימות למשתמש.

פעולה 5 בתפריט: **Display all stations** – מדפיסה את נתוני כל התחנות.

יש לכתוב פונקציה בשם **dispAllSt**. הפונקציה מקבלת מבנה מתנהל של תחנות הטעינה.

הפונקציה תדפיס את הנתונים הבאים של כל התחנות: שם, id, פירוט עמדות טעינה, מספר רכבים שמחוברים לטעינה ומספר רכבים שמחכים בתור.

פעולה 6 בתפריט: **Display cars at station** – מדפיסה את נתוני הרכבים שנמצאים בתחנה.

יש לכתוב פונקציה בשם **dispCarsAtSt**. הפונקציה מקבלת מבנה מתנהל של תחנות הטעינה.

הפונקציה תקלוט מהמשתמש את שם התחנה או את מספר ה-id שלה ותדפיס את נתוני הרכבים הנמצאים בתחנה. יש להפריד בין רכבים שנמצאים בטעינה לרכבים שנמצאים בתור. יש להדפיס את הנתונים הבאים: מספר רישוי, סוג טעינה מתאים, ועבור רכבים הנמצאים בטעינה יש להדפיס את משך הטעינה עד כה בדקות.

פעולה 7 בתפריט: **Report of stations' statistics** – מדפיסה דו"ח על נתוני התחנה.

יש לכתוב פונקציה בשם **reportStStat** המקבלת מבנה מנהל של תחנות הטעינה.

הפונקציה תקלוט מהמשתמש את שם התחנה או את מספר ה-id שלה ותדפיס עבורה:

- שיעור ניצול עמדות טעינה באחוזים השווה למספר עמדות טעינה תפוסות חלקי סך תחנות הטעינה.
- אחוז עמדות טעינה תקולות (מספר עמדות תקולות חלקי סך עמדות טעינה)
- **רמת עומס** בתחנה הנמדדת ע"י עומס יחסי בין מספר הרכבים כרגע בעמדות טעינה למספר הרכבים שמחכים בתור. יש לציין גם על פי מדד זה אם **אין עומס** (המדד גדול מ- 1 או שהתור ריק), **מצב מאוזן** (~1), **עמוס** (קטן מ־1), **עומס כבד מאוד** (קטן מ- 0.2).

פעולה 8 בתפריט: **Display Top customers** – מדפיסה 5 רכבים בעלי ההוצאות הגבוהות ביותר.

יש לכתוב פונקציה בשם **dispTopCustomers** המקבלת מבנה מנהל של BST מכוניות, ותדפיס את חמשת הרכבים בעלי ההוצאות הגבוהות ביותר: מספר רישוי והתשלום הכולל עד היום.

פעולה 9 בתפריט: **Add new port** – מוסיפה עמדת טעינה חדשה בתחנה מסוימת.

יש לכתוב פונקציה בשם **addNewPort** המקבלת מבנה מנהל של תחנות הטעינה.

הפונקציה תקלוט מהמשתמש את שם התחנה או את מספר ה-id שלה וקצב טעינה של העמדה להוספה ותקים את העמדה החדשה בתחנה. במידה ויש מכוניות מתאימות בתור, הפונקציה תקדם את התור ותחבר את הרכב המתאים לטעינה. יש להדפיס הודעות מתאימות למשתמש.

פעולה 10 בתפריט: **Release Charging Ports** – הפסקת טעינה של רכבים בתחנה שנבחרה.

יש לכתוב פונקציה בשם **releasePorts**. הפונקציה מקבלת מבנה מנהל של עץ תחנות הטעינה.

הפונקציה תקלוט מהמשתמש את מספר התחנה ותנתק רכבים שנמצאים בטעינה מעבר ל- 10 שעות.

הפונקציה תדפיס למשתמש את מספרי הרכבים שנותקו ותקדם את התור במידת הצורך. הפונקציה תודיעה אם התחברו מכוניות חדשות לעמדות טעינה.

פעולה 11 בתפריט: **Remove out of order ports** – מוחקת עמדת טעינה תקולה.

יש לכתוב פונקציה בשם **remOutOrderPort**. הפונקציה מקבלת מבנה מנהל של עץ התחנות.

הפונקציה תקלוט מהמשתמש את מספר התחנה (id). תדפיס את רשימת עמדות האענה התקולות ותבקש מהמשתמש להכניס את מספר עמדת הטעינה התקולה. הפונקציה תסיר את עמדת הטעינה התקולה מהתחנה ותדפיס הודעה למשתמש אם ההסרה הצליחה.

פעולה 12 בתפריט: **Remove customer** – מחיקה של רכב (לקוח).

יש לכתוב פונקציה בשם **remCustomer**. הפונקציה מקבלת מבנה מנהל BST של מכוניות.

יש לקלוט מהמשתמש מספר רכב ולמחוק אותו. יש לשחרר זיכרון. שימו לב כי לא ניתן למחוק רכב שנמצא בתור או בטעינה. יש להדפיס הודעה למשתמש אם הפעולה הצליחה ולדאוג לעדכון של כל מבני הנתונים הרלוונטיים.

פעולה 13 בתפריט: **Close station** – סוגרת את התחנה לפי בקשת המשתמש.

יש לכתוב פונקציה בשם **closeSt** – הפונקציה מקבלת מבנה מנהל של תחנות הטעינה.

הפונקציה תקלוט מהמשתמש את התחנה (id או שם) ותסגור את התחנה ע"י שחרור זיכרון בהתאם לצורך ותעדכן את כל מבני הנתונים הרלוונטיים. יש להדפיס הודעות מתאימות.

פעולה 0 בתפריט: **Exit System** – יציאה מהמערכת.

יש לכתוב פונקציה בשם **exitSys** – הפונקציה מקבלת מבנה מנהל של התחנות ומבנה מנהל של הרכבים וסוגרת את המערכת לאחר שמירה של כל הנתונים לקבצים. יש לדאוג לשחרר זיכרון ולהדפיס הודעה שהמערכת נסגרה בהצלחה.

הנחיות כלליות ליישום

- לכל טיפוס של נתונים יש להתייחס כאל ADT ולכתוב סט של פונקציות לניהולם שישמרו בספרייה ייעודית: אתחול, הוספה, מחיקה, הדפסה.
- יש לעבוד בסביבה מרובת קבצים
- יש לדאוג לקוד יעיל ליישום הפונקציות
- יש להשתמש בהגדרת קבועים ופקודות מאקרו במידת הצורך
- יש לדאוג לממשק נוח וברור למשתמש העובד עם המערכת
- יש לרשום תיעוד מפורט של הקוד **באנגלית**
- יש לדאוג לשחרר כל זיכרון דינאמי שאינו בשימוש + שחרור מסודר לפני היציאה מהמערכת

דוגמא למעטפת כללית לקוד של התוכנית הראשית:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

// Include your libraries

// Structures and function prototypes go here

int main() {
    // Initialize data structures and other necessary variables

    // Display menu and handle user input
    return 0;
}

// definition of functions goes here
```

מדדים להערכה של העבודה

ציון העבודה יקבע על פי המדדים הבאים:

נכונות הקוד – יש לוודא שהפעולות אותם אתם מבצעים על הנתונים לא פוגעים בנכונות הנתונים וישנו טיפול קפדני בכל השגיאות שעלולות להופיע מבלי שתהיה קריסה של המערכת. יש לוודא שכל פעולה מתבצעת באופן מלא ותקין. דו"ח בדיקות אותו אתם מגישים (ראו הנחיות בסוף) אמור לכסות את כל המקרים האפשריים עם הסברים מפורטים של מה נבדק.

הקוד חייב לרוץ בסביבת VS Community 2022 בה נבדקות העבודות. קוד שלא ירוץ, לא ייבדק, והציון בעבודה יהיה נכשל.

שמות השדות, הטיפוסים והחתימות של הפונקציות – יש להיצמד לשמות שהוגדרו בעבודה.

יעילות הקוד – יש לכתוב קוד לטיפול יעיל בנתונים בהתאם לסוג מבנה הנתונים איתו אתם עובדים

מודולריות הקוד – יש לחלק את הקוד לפונקציות המותאמות לכל סוג של נתונים

נוחות ממשק משתמש – יש לספק אפשרויות ברורות למשתמש לאינטראקציה נוחה בניהול בסיס הנתונים

עקביות המערכת – יש לדאוג שהנתונים נשמרים ונטענים באופן תקין בין הרצה להרצה של המערכת ולוודא שהמערכת מטפלת בכל המקרים כך שלא תתרחש קריסה בגלל שגיאות מכל סוג שהוא

תיעוד – יש להוסיף הערות מלאות וברורות בקוד המסבירות את המטרה והפונקציונליות של כל פעולה ופונקציה באנגלית

עמידה בכללי ההגשה – יש להגיש את העבודה על פי כללי ההגשה המופיעים למטה

קובץ בדיקות – יש לכלול בסוף הקובץ להגשה בדיקות שבוצעו למערכת, שיכלול הסבר על הבדיקות השונות, צילום של הקלט והפלט בעת הבדיקה. יש לבצע בדיקה שכל הפונקציות שהוגדרו.

שימוש ב- ADT וסביבת מרובת קבצים - יש לעבוד בסביבה מרובת קבצים למימוש מבני הנתונים השונים.

כללי ההגשה

במהלך התקופה שניתנה לביצוע העבודה, יהיה עליכם להגיש 2 גרסאות של עבודה בהתאם להנחיות המופיעות באתר הקורס ולהספק שלכם (גרסה אמצע וגרסה סופית) של עבודתכם בתאריכים ידועים מראש. הגשות אלה הן חובה. **סטודנטים אשר לא יגישו את כל קבצי הביניים, יקבלו ציון נכשל בעבודה גם אם גרסתם הסופית תהיה מושלמת.**

יש לשים לב להנחיות ההגשה של כל תיבה. תהיה הורדה בניקוד במידה ולא תוגש העבודה על פי ההנחיות ועם הקבצים המתאימים.

בכל גרסה של עבודה – חלקית או סופית (סה"כ 2 הגשות בתאריכים שונים), יש להגיש לשתי תיבות הגשה:

- **תיבת הגשה 1:** יש להגיש קובץ ZIP או RAR המכיל את התוכנית במלואה (ספריית הפרויקט ב-VS), כולל כל הקבצים והספריות שיצרתם + קבצי הטקסט עם הנתונים בהם השתמשתם (Stations.txt, Cars.txt, Ports.txt ו- LineOfCars.txt), ושידרשו להרצה תקינה של התוכנית. אנא מקמו את הקבצים בספריית הפרויקט על מנת לאפשר הרצה תקינה.
- **תיבת הגשה 2:** יש להגיש קובץ WORD בו יופיעו **קבצי הקוד במלואם עם ההזחות המתאימות** באופן ברור וקריא (חשבו על הבודק/ת ונוחות הבדיקה. עבודה נוחה לבדיקה = בודק/ת נחמד/ה יותר). בסוף קובץ WORD, יש לצרף את **דו"ח הבדיקות של הקוד**. הדו"ח יכיל פירוט מילולי של הבדיקות שבוצעו + צילומי מסך של הקלט והפלט הרלוונטיים עבור כל בדיקה (גם פלט לקבצים). על הבדיקות להיות כמה שיותר מקיפות.

הערות חשובות:

אנו מעודדים אתכם להרחיב את העבודה בהתאם לסקרנות והצרכים שלכם.

אנו מעודדים אתכם להתייעץ עם חברים לכיתה, לחלוק ביניכם רעיונות ולהתייעץ בפתרון בעיות, אך כתיבת העבודה הינה אישית וקטעי קוד זהים אינם מקובלים.

אין להעתיק עבודות או להיעזר באמצעים חיצוניים כגון תשלום לגורם אחר. שימו לב שייתכן מאוד שתתבקשו להגן על עבודתכם.

סטודנטים שעבודתם תיחשד כמועתקת או שלא בוצעה על ידם, גם אם באופן חלקי, יעמדו לדין משמעתי והקורס יפסל.